Sandbox create path — how long each step takes

Daytona API, JetStream, runner, Firecracker and guest, per sandbox, on HQ Vera, EPYC 9575F, Turin and Graviton5

Benchmark runs of 2026-10-14 (same build `f0611fa13` + bench patches on every host); fixes = 3260 init-disk no-fsync, 3261 cgroup pool, 3265 permanent neighbour, 3269 API backlog 4096.

1. Summary

One sandbox on an idle host takes ≈ 85–90 ms from `POST /vms` to the first command answered (HQ Vera, server long-poll client): API ≈ 5 ms, runner ≈ 60 ms (Firecracker spawn 39–53 ms is the largest piece, snapshot load 4–6 ms), event back to the client ≈ 2 ms, first command through the proxy ≈ 21 ms. The benchmark harness reports ≈ 275 ms for the same thing, because its SDK polls for `running` (first poll after 250 ms).

Under a 1,000-sandbox burst the time is spent waiting, not working. With the fixes on, the typical sandbox spends 0.3–0.9 s waiting for a runner slot (step B), 0.25–0.7 s in the runner clone (D) and 0.2–0.35 s for the harness to notice it is running (E). The API itself takes ≈ 0.1 s of a burst sandbox's time.

The dominant step differs per host (1k burst, idle image, fixes on, p50):

- **HQ Vera:** queue for a runner slot 715 ms (256 in-flight cap), Firecracker spawn **208 ms** — 2.5× the other hosts.
- **EPYC 9575F:** API answering the POST 652 ms (all 1,000 accepted at once on 128 threads); clone 413 ms.
- **Graviton5:** the fastest clone (246 ms) and total (0.89 s).
- **Turin:** snapshot load + resume **484 ms** vs 16–39 ms elsewhere — its single biggest step, unexplained so far.

Without the fixes, the init-disk copy + fsync dominates the clone: 1,293 ms on the 9575F, 567 ms on Graviton5, 352 ms on Vera ⟶ 11–38 ms with the fix. **With the agent image** every runner step is CPU-starved (Firecracker spawn 1.7–1.8 s on Vera and the 9575F); Turin's clone stays at 1.2 s.

2. The steps

Step	Component	What happens	Where it is measured
A	client ⟶ API	`POST /vms`: TCP accept, auth, audit row, validation and DB lookups, `BEGIN`, insert `vms` + `jobs` rows, publish the create job to JetStream (waits for the store ack), `COMMIT`, HTTP 200	client: POST sent ⟶ answered
B	API ⟶ JetStream ⟶ runner	Job waits in the JetStream `JOBS` consumer until the runner has a free slot (`MaxAckPending` = 256 in flight by default; 2,048 on the 9575F), then the runner handler starts (semaphore 512)	client send time ⟶ runner `boot-phases create` log line
C	runner	Admission / capacity reserve, image resolve, manifest + template lookup	runner journal
D1	runner	Take a pre-built network namespace from the pool	`netns_get_ms`
D2	runner	Copy the scratch file and a private 16 MiB init disk, `fsync` it (3260 `template.go`, `vm.go`)	`scratch_cp_ms`
D3	runner + kernel	Fork `rlp-fcns` (setns, join cgroup), exec Firecracker, wait for its API socket (3212: 30 s wait)	`fc_start_ms`
D4	Firecracker	Load the template snapshot (memory file mapped, device state) and resume the vCPU	`load_ms`
D5	runner ⟶ guest	TCP-probe the guest daemon on :2280 until it accepts (the arm64 580 ms ARP stall lives here, 3265)	`toolbox_wait_ms`
D6	runner + kernel	Everything else in the clone: per-VM cgroup `mkdir` (3261), register the instance, entrypoint trigger	clone total − D1…D5
E	runner ⟶ NATS ⟶ API ⟶ client	`running` event published, API overlay updated, client learns the state. Long-poll client: 2–3 ms. Harness SDK: polls at 0.25 s ×1.5 up to 2 s, so this step is mostly poll interval	`running-emit` ⟶ client sees `running`
F	client ⟶ proxy ⟶ guest	First command (`echo ready`) through the toolbox proxy to the guest daemon	client

Method (sections 4–6). Every host runs client, API, NATS, runner and guests on one machine, so all timestamps share one clock. Each sandbox in the harness runs (`mpfleet.py`, 16 client processes) was joined by `vm_id` with the runner's journal (`boot-phases create`, `boot-phases`, `running-emit` lines, pulled read-only from each host for the exact run windows). Values are pooled over all reps, ms, **p50 / p90**. **† Step A overlaps step B**: the job is published before the API commits and answers, so on the 9575F the runner often starts before the client gets its 200. Steps B, C, D, E, F are sequential; medians of steps do not add up exactly to the median total. Turin has fixes-on data only (its fixes-off runner journal rotated out); Turin used a 512-connection client pool and 64 MiB disks, the others 1,024 and 1 GiB.

3. One sandbox at a time (low load)

3.1. HQ Vera, server long-poll client, all sources joined (API audit + DB, NATS tap, runner journal)

Step (ms)	p50	p90	with all 4 fixes: p50 / p90 / p99
1 client → API (HTTP, auth, audit)	1.2	1.3	
2 API handler before the DB transaction	2.4	2.7	
3 API DB insert + JetStream publish	1.1	1.4	
4 API commit + HTTP response	0.6	0.8	
5 queue: JetStream → runner handler	0	0.3	
6–8 runner admission, image resolve, manifest + template lookup	1	1	
9a network namespace from pool	0	0	
9b scratch copy + 16 MiB init-disk copy + fsync	7	8	
9c Firecracker spawn (incl. cgroup join) + API socket	**39**	45	
9d snapshot load + resume	6	7	
9e guest toolbox port accepts	2	**580**	max 2 ms (stall gone, 3265)
9f other: cgroup mkdir, register, entrypoint trigger	3	4	
10 running event → NATS	0.9	1.3	
11 API sees running → long-poll answers client	1.2	1.6	
POST → client sees `running`	**65**	**644**	**64 / 69 / 73**
12 first exec through the proxy (`echo ready`)	21	26	
POST → first exec done	**88**	**662**	≈ 85

200 sequential creates (control) and 60 sequential creates per fix arm. 23 % of control creates stall ≈ 580 ms in step 9e (lost first ARP after resume on arm64); a permanent neighbour entry removes it. Source: `CREATE-PATH-STEPS.md`.

3.2. Harness client, 10 sandboxes at once (warm-up runs), all hosts with runner data

Step (ms, p50 / p90)	HQ Vera off	HQ Vera on	EPYC 9575F off	EPYC 9575F on	Graviton5 off	Graviton5 on
A · API accepts the create (POST answered)†	16 / 19	17 / 19	17 / 20	17 / 19	19 / 48	25 / 32
B · request sent → runner starts the job (API + JetStream queue)	7 / 8	6 / 7	7 / 9	7 / 8	7 / 16	12 / 13
C · runner admission, image + template lookup	1 / 1	0 / 1	1 / 1	0 / 1	1 / 1	1 / 1
D1 · network namespace from the pool	0 / 0	0 / 0	0 / 2	0 / 0	0 / 1	0 / 0
D2 · scratch copy + 16 MiB init disk (fsync)	31 / 31	5 / 5	38 / 41	7 / 8	30 / 32	10 / 13
D3 · Firecracker spawn + API socket	84 / 89	53 / 56	7 / 17	18 / 20	5 / 28	27 / 28
D4 · snapshot load + resume	4 / 6	4 / 5	6 / 8	9 / 11	12 / 58	6 / 7
D5 · guest toolbox port accepts	2 / 3	3 / 3	16 / 17	16 / 17	2 / 22	2 / 2
D6 · cgroup create/join, register, entrypoint trigger	8 / 35	3 / 9	13 / 15	3 / 5	21 / 24	2 / 21
D · runner clone total	131 / 135	68 / 70	81 / 83	52 / 58	74 / 119	48 / 52
E · running event → client sees it (NATS → API → SDK poll)	181 / 193	196 / 199	182 / 191	211 / 216	193 / 215	221 / 229
F · first command through the proxy (`echo ready`)	2 / 3	2 / 3	2 / 3	2 / 3	3 / 13	2 / 3
Total: request → first command done	273 / 274	274 / 275	273 / 275	273 / 274	276 / 318	279 / 294
samples	20	20	20	20	40	20

The harness total is ≈ 275 ms on every host, fixes or not: step E (≈ 180–220 ms) is the SDK's first status poll at 250 ms, not platform time. The runner clone is 48–68 ms with the fixes (Vera's Firecracker spawn is the slowest piece, 53 ms).

4. 1,000 sandboxes at once — idle image (`alpine:3.20`)

Step (ms, p50 / p90)	HQ Vera off	HQ Vera on	EPYC 9575F off	EPYC 9575F on	Graviton5 off	Graviton5 on	Turin on
A · API accepts the create (POST answered)†	324 / 1,208	285 / 438	349 / 1,169	652 / 774	143 / 208	196 / 334	252 / 340
B · request sent → runner starts the job (API + JetStream queue)	1,271 / 2,244	715 / 1,493	268 / 1,894	411 / 760	756 / 1,855	321 / 748	865 / 2,009
C · runner admission, image + template lookup	1 / 1	1 / 2	1 / 2	1 / 12	1 / 1	1 / 2	1 / 1
D1 · network namespace from the pool	0 / 0	0 / 0	0 / 0	0 / 6	0 / 0	0 / 0	0 / 0
D2 · scratch copy + 16 MiB init disk (fsync)	352 / 665	11 / 33	1,293 / 1,605	38 / 79	567 / 602	14 / 36	10 / 56
D3 · Firecracker spawn + API socket	56 / 143	208 / 320	10 / 27	83 / 193	5 / 16	83 / 136	26 / 105
D4 · snapshot load + resume	11 / 51	32 / 110	8 / 23	39 / 172	5 / 17	16 / 66	484 / 1,196
D5 · guest toolbox port accepts	3 / 9	6 / 510	17 / 19	18 / 30	4 / 22	28 / 43	16 / 19
D6 · cgroup create/join, register, entrypoint trigger	151 / 370	124 / 273	12 / 75	155 / 339	3 / 28	78 / 162	17 / 100
D · runner clone total	629 / 1,129	441 / 822	1,341 / 1,720	413 / 640	593 / 675	246 / 349	706 / 1,277
E · running event → client sees it (NATS → API → SDK poll)	421 / 805	319 / 686	474 / 909	220 / 408	282 / 1,026	182 / 476	346 / 1,025
F · first command through the proxy (`echo ready`)	4 / 7	4 / 9	3 / 7	5 / 11	5 / 24	15 / 48	4 / 8
Total: request → first command done	2,362 / 3,693	1,591 / 2,553	2,522 / 3,835	1,076 / 1,559	2,170 / 3,475	889 / 1,526	2,353 / 3,734
samples	3,000	2,999	3,000	3,000	3,000	3,000	2,000

Reading.

- **Fixes off:** D2 (init disk + fsync) is the largest runner step on every host — 1.3 s on the 9575F, 0.57 s on Graviton5, 0.35 s on Vera. Vera additionally spends 1.27 s queued for a runner slot (B).
- **Fixes on:** D2 drops to 10–38 ms. What remains: the queue (B: 0.3–0.9 s), Firecracker spawn (D3: Vera 208 ms, others 26–83 ms), cgroup/register (D6: 17–155 ms) and, on Turin only, snapshot load (D4: 484 ms).
- The API share (A) is 0.2–0.3 s except on the 9575F (0.65 s), where all 1,000 POSTs are accepted at once and compete for 128 threads; that does not delay the runner, which already has the jobs (B = 0.41 s).

4.1. HQ Vera deep trace, 1,000-sandbox burst (long-poll client) — which fix moves which step

Step (ms, p50 / p90)	control	+init-disk nofsync	+cgroup pool	+API backlog	all four
1 client → API	40 / 1,072	41 / 1,066	49 / 1,077	34 / 51	**32 / 61**
2 API handler before DB txn	18 / 80	13 / 22	30 / 35	87 / 451	45 / 72
3+4 API DB insert, publish, commit, response	10 / 35	13 / 39	24 / 37	14 / 122	13 / 55
5 queue → runner handler	793 / 1,247	337 / 822	629 / 964	1,214 / 2,180	**421 / 1,172**
9b scratch + init-disk copy	300 / 700	**9 / 20**	516 / 689	126 / 959	**11 / 35**
9c Firecracker spawn + API socket	63 / 180	106 / 219	60 / 137	56 / 144	200 / 289
9d snapshot load + resume	12 / 69	39 / 148	11 / 39	94 / 381	20 / 58
9e guest toolbox accepts	3 / 11	4 / 478	3 / 14	4 / 462	5 / 76
9f cgroup mkdir, register, trigger	201 / 381	427 / 565	**24 / 58**	123 / 495	139 / 269
11 API → long-poll answers client	3 / 89	2 / 4	61 / 250	2 / 65	1 / 4
POST → client sees running	**1,927 / 3,035**	1,600 / 2,709	1,725 / 2,653	2,296 / 3,370	**1,054 / 1,748**
listen-queue overflows per burst	424–687	460	656	0	0

Each fix alone moves 10–20 %; together −45 %. Removing one bottleneck hands the time to the next (the backlog fix alone is slightly worse end-to-end: requests stop waiting on TCP retries and wait in the runner queue instead).

5. 1,000 sandboxes at once — agent image (startup under CPU load)

Step (ms, p50 / p90)	HQ Vera off	HQ Vera on	EPYC 9575F off	EPYC 9575F on	Graviton5 off	Graviton5 on	Turin on
A · API accepts the create (POST answered)†	283 / 455	322 / 506	381 / 446	899 / 1,118	124 / 182	228 / 381	258 / 368
B · request sent → runner starts the job (API + JetStream queue)	4,187 / 13,736	2,446 / 18,456	230 / 5,126	563 / 4,133	949 / 4,419	685 / 5,067	970 / 3,102
C · runner admission, image + template lookup	1 / 2	1 / 2	1 / 1	1 / 5	1 / 1	1 / 2	1 / 1
D1 · network namespace from the pool	0 / 0	0 / 0	0 / 0	0 / 0	0 / 0	0 / 0	0 / 1
D2 · scratch copy + 16 MiB init disk (fsync)	218 / 741	14 / 133	1,008 / 1,578	54 / 143	299 / 612	12 / 34	17 / 72
D3 · Firecracker spawn + API socket	1,335 / 4,931	1,677 / 11,007	468 / 3,526	1,799 / 5,026	44 / 1,170	491 / 2,421	74 / 200
D4 · snapshot load + resume	211 / 2,373	36 / 650	114 / 257	130 / 301	30 / 606	30 / 476	866 / 1,981
D5 · guest toolbox port accepts	11 / 512	5 / 513	107 / 380	131 / 402	21 / 142	19 / 531	24 / 115
D6 · cgroup create/join, register, entrypoint trigger	1,121 / 4,832	372 / 2,929	2,976 / 8,759	1,736 / 7,021	565 / 2,726	561 / 2,499	49 / 161
D · runner clone total	5,232 / 9,350	4,318 / 12,153	6,237 / 11,677	5,487 / 11,296	1,655 / 3,952	2,050 / 4,066	1,188 / 2,204
E · running event → client sees it (NATS → API → SDK poll)	649 / 1,772	859 / 1,523	765 / 1,700	625 / 1,717	556 / 1,705	479 / 1,425	391 / 1,328
F · first command through the proxy (`echo ready`)	9 / 37	11 / 41	51 / 228	54 / 236	54 / 296	38 / 252	13 / 70
Total: request → first command done	11,846 / 21,834	11,725 / 23,940	7,949 / 17,934	6,765 / 16,566	3,558 / 9,469	5,506 / 9,626	2,556 / 5,767
samples	2,998	2,998	3,000	3,000	3,000	3,000	2,000

Reading. Every sandbox starts a ≈ 3–4 core-second job at boot, so the runner competes with the guests for CPU. Firecracker spawn (D3) and cgroup/register (D6) balloon to seconds on Vera and the 9575F; queue waits (B) reach 2–4 s on Vera. Turin (384 threads, fast cores) keeps the whole clone at ≈ 1.2 s and the total at 2.6 s; its D4 snapshot load (0.87 s) is again the largest piece. The fixes do not help here: faster early steps only start the guests' jobs sooner.

6. 2,000 sandboxes at once (fixes on)

Step (ms, p50 / p90)	HQ Vera on	EPYC 9575F on	Turin on	HQ Vera on	EPYC 9575F on	Turin on
A · API accepts the create (POST answered)†	727 / 1,068	1,257 / 1,603	652 / 1,079	952 / 1,809	2,398 / 2,942	793 / 1,327
B · request sent → runner starts the job (API + JetStream queue)	1,858 / 4,106	868 / 2,357	2,284 / 4,565	15,616 / 40,159	5,957 / 25,924	3,946 / 11,721
C · runner admission, image + template lookup	1 / 1	1 / 2	0 / 1	1 / 3	1 / 1	1 / 1
D1 · network namespace from the pool	0 / 0	0 / 0	0 / 0	0 / 0	0 / 0	0 / 11
D2 · scratch copy + 16 MiB init disk (fsync)	16 / 63	21 / 61	10 / 36	29 / 761	72 / 194	42 / 98
D3 · Firecracker spawn + API socket	188 / 424	82 / 198	28 / 67	3,625 / 8,509	3,052 / 9,262	296 / 1,885
D4 · snapshot load + resume	37 / 335	152 / 442	657 / 1,248	140 / 1,791	430 / 1,064	255 / 1,912
D5 · guest toolbox port accepts	11 / 523	32 / 517	17 / 20	11 / 550	271 / 940	82 / 341
D6 · cgroup create/join, register, entrypoint trigger	104 / 284	85 / 330	9 / 77	694 / 2,855	5,984 / 16,766	249 / 2,101
D · runner clone total	589 / 1,098	700 / 1,244	789 / 1,316	6,285 / 11,616	11,483 / 23,433	2,189 / 4,166
E · running event → client sees it (NATS → API → SDK poll)	413 / 1,415	354 / 849	509 / 1,281	930 / 1,795	868 / 1,800	692 / 1,685
F · first command through the proxy (`echo ready`)	4 / 9	6 / 12	5 / 11	10 / 39	125 / 642	29 / 148
Total: request → first command done	2,965 / 6,315	2,403 / 3,486	3,496 / 6,754	27,620 / 46,069	19,879 / 50,431	7,073 / 17,184
samples	4,000	6,000	4,000	3,998	4,000	2,000

Reading. At 2,000 the queue for a runner slot (B) becomes the largest step everywhere: 0.9–2.3 s idle, 4–16 s with the agent image. The 9575F (in-flight cap 2,048) has the shortest idle queue; Vera and Turin run the default 256.

7. What to take from this

Finding	Detail
The platform's own work is small	≈ 60 ms of runner work and ≈ 5 ms of API per sandbox when idle; under a burst the API share is ≈ 0.1 s (long-poll client) and the rest is queueing behind serialized kernel work and the 256-slot cap.
The harness adds a fixed ≈ 0.2–0.35 s	Step E is SDK polling (0.25 s ×1.5 up to 2 s); the long-poll client sees `running` in 1–3 ms. Compare hosts with the same client, and use long-poll for absolute numbers.
Vera's slow step: Firecracker spawn	208 ms p50 in a burst (53 ms idle) vs 26–83 ms on the other hosts. Includes forking `rlp-fcns`, joining the cgroup and waiting for the API socket. Worth a per-step trace on Vera (fork/exec, cgroup join, page-table setup on 64 KiB pages).
Turin's slow step: snapshot load	484 ms p50 idle burst (0.87 s agent) vs 16–39 ms elsewhere on the same build. Not yet explained; candidates: snapshot memory file read path on the single NVMe / page cache, Firecracker's x86 restore cost at this concurrency.
Queue cap differs across hosts	`RLP_VM_MAXACK` 2,048 on the 9575F vs 256 on Vera, Turin and Graviton5. It sets step B; equalize before comparing burst startup across hosts.
Agent-image numbers measure CPU	All runner steps stretch with guest CPU load; Turin's lead there is CPU throughput, not a faster create path.

Data: per-run client JSONL from `vera-hq-cell-20261014/{fixes-bench,epyc9575-fixes-bench,run2k-*}.tgz`, `graviton5-cell-20261014/fixes-bench.tgz` (first attempt's ON arm excluded: only the API flag applied), `phoenix-e6-cell-20261014/results-fixed.tgz`; runner journals pulled read-only for the exact run windows (`create-path-breakdown/runner-journals.tgz`); join and statistics in `create-path-breakdown/analyze.py` → `agg.json`, `agg-warm.json`; tables generated by `gen_tables.py`. Deep Vera trace: `vera-hq-cell-20261014/CREATE-PATH-STEPS.md`.